Divergence-form and wide-halo mEVP in the GPU version of FESOM2: implementation and measurements

Nikolay Koldunov

6 August 2026

1. Scope

Two of your proposals were implemented in our GPU version of FESOM2 and measured: **(a)** the **reformulation of the mEVP sub-cycling** in which the quantity carried from one sub-cycle to the next is the divergence of the stress, held at nodes, instead of the three stress components held at elements; and **(b)** the **wide halo**, in which the halo is exchanged only every K-th sub-cycle and the missing information is recomputed locally in a ring of ghost cells K deep, so that the result is identical to exchanging after every sub-cycle.

The main finding is about neither proposal directly. **How the ring computation is written matters more than what it carries.** Written as separate kernels for the ring — which is how we first implemented it — the wide halo removes 10 to 28% of the sea-ice cost. Written as a widening of the loops that already exist, which is how your reference code does it, the same scheme removes 37 to 58%. Our first set of conclusions, including a negative one about your reformulation, was measuring our implementation.

Every timing is given twice, against the model step and against the ice cost, because sea ice is 5 to 13% of a step and the step divides an ice-only change by about ten.

2. Configurations, meshes, machine

mEVP advances the ice momentum with 120 sub-cycles per model step in all our configurations. In the standard scheme the ice-velocity halo is exchanged after every one of them.

name	**carried between sub-cycles**	**halo**
standard	stress $\sigma_{11}, \sigma_{12}, \sigma_{22}$ at elements	exchanged every sub-cycle
divergence form	$\nabla\cdot\sigma$ (two components) at nodes	exchanged every sub-cycle
wide halo, standard form	stress at elements, ring K deep	every K-th, *exact*
wide halo, divergence form	$\nabla\cdot\sigma$ at nodes, ring K deep	every K-th, *exact*

Each wide-halo configuration exists in two *writings* of the same scheme, which is the axis this report is really about: **ring kernels** (a separate GPU kernel per stage per sub-cycle for the ghost ring) and **widened loops** (the ring folded into the owned loop it duplicates). The two compute the same ring and put identical traffic on the wire.

All four meshes are production configurations. Levante at DKRZ: GPU nodes carry four NVIDIA A100 (80 GB), one MPI process per GPU; CPU nodes two AMD EPYC 7763, 128 cores each. Every comparison is made inside one job allocation with the same executable and is the faster of two repetitions; repetition spread is below 0.4% on GPU, about 1.5% on CPU.

	surface nodes	**elements**	**levels**	**time step**
CORE2	126 858	251 746	48	1800 s
fArc	638 387	1 270 173	48	900 s
DARS	3 160 340	6 294 801	57	120 s
NG5	7 402 886	14 827 217	70	180 s

3. Why the sub-cycle is expensive

One model step performs 120 ice-velocity halo exchanges, each moving very little data, with very little arithmetic between them. Koldunov et al. (2019, GMD 12, 3991) diagnosed this on fArc and proposed, without testing it, "wider halo regions to reduce communication frequency".

On CPU, at 2304 cores on fArc, the ice is 13.2% of the step, and inside the ice dynamics the time spent *waiting* for MPI is 4.8 ms per step against 3.5 ms of computation. The cost is idle time.

On GPU there is a second reason. Each exchange is a sequence of small GPU operations, each carrying a fixed issue cost independent of how little data it touches. On CORE2 with 8 GPUs the ice-dynamics phase takes 8.2 ms per step, of which about 1.25 ms is arithmetic, spread over some 480 separate operations. The sub-cycle is limited by how often work is dispatched, not by arithmetic or memory traffic — which is the single fact behind most of what follows, including the results that came out negative.

4. What was implemented, and how it was checked

All configurations are selected at run time and are off by default.

The divergence form. The stress recursion is replaced by a recursion for $R = \nabla\cdot\sigma$ at nodes, with the ice-element mask applied so that R advances only where ice elements contribute.

The wide halo. At the start of each model step one extended exchange fills a ring of ghost nodes and elements K cells deep with the owner's values of the eleven fields the sub-cycle needs. Over the next $K-1$ sub-cycles each process advances that ring itself and needs no communication; on the K-th the ring is refreshed. Ring geometry, boundary mask and per-step element quantities are shipped from their owners rather than recomputed, because a locally recomputed value can differ in the last bit and that is enough to destroy exactness.

Widened loops. Each ring kernel is folded into the owned loop it duplicates, selecting the ghost arrays by an index branch, and — the two being one loop thereafter — the ring nodes are assembled by the same scatter the owned nodes use rather than by replaying the owner's summation order. That last step gives up bit-identity deliberately; Sect. 5.5 is the test that replaces it.

Correctness. For the exact writings the requirement is strict: with the option on the model must produce bit-for-bit the same output as with it off. Verified for both forms at $K = 4$ and $K = 8$, and separately for the same machinery under standard EVP. All options were shown to leave the model unchanged when a different vertical mixing scheme or vertical coordinate is selected, and the GPU version was checked against the CPU version with the options on.

5. Results

5.1 The reformulation is exact

Carrying R and σ simultaneously and comparing them sub-cycle by sub-cycle, R agrees with $\nabla\cdot\sigma$ to 1–2.5×10^{-15} relative, and this does not grow over 20 model steps, i.e. 2400 sub-cycles.

One genuine non-equivalence is worth naming. Within a step the ice-element mask is fixed and the two forms are equivalent; *across* steps, when an element enters or leaves the mask, they are not, because the recursion has been applied after summation around a node rather than before. It fires only on steps where mask membership changes and is bounded by 2.1×10^{-11} relative — far below physical significance, but it means the divergence form should be introduced as a scheme change rather than as an optimisation.

5.2 What the reformulation costs or saves by itself

Measured against the ice cost the reformulation **does** pay on CPU, and more so the more processes are used: −2.2% at 512 processes on fArc, −3.2% at 1024, −5.6% at 1536, −6.7% at 2048, and −8.5% on CORE2 at 512. Against the model step the same runs read −0.5% to 0%, which is why we first reported it as nothing. On GPU it is a *cost* in both currencies, +2 to +17% of the ice.

The reason is Sect. 3. To bound the largest saving it could possibly deliver we ran a control: three element-sized arrays that the mEVP path writes every sub-cycle and never reads were simply not written. That removes strictly more memory traffic than the reformulation does, and it changed the run time by **exactly 0.00%**. Total GPU arithmetic time is identical in the two forms, 21.8 against 21.7 ms per step. There was no memory-traffic saving available to collect.

5.3 The wide halo, and the fact that dominated it

As we first wrote it, with a ring kernel per stage per sub-cycle, the wide halo removes 10 to 28% of the ice cost at the four operating points, and nothing at all on CPU.

Before changing anything we profiled why. Our ring kernels add **362 GPU kernel launches per model step**, and each is on a slower launch path than the owned kernel it duplicates: the ring kernels capture enough array descriptors to cross a threshold in Kokkos above which arguments are staged through constant memory rather than passed with the launch. The measured gap between consecutive launches is 14.3 μs for those against 4.4 μs for the owned kernels, and over a step the ring kernels cost 5.9 ms of inter-kernel idle against 2.5 ms of arithmetic. The launch structure was more than twice the computation it carried.

Written as widened loops the same scheme, computing the same ring and refreshing it on the same schedule, gives:

change of the sea-ice cost at K=8, each mesh at its operating point	CORE2 1 node / 4 GPUs	fArc 4 / 16	DARS 8 / 32	NG5 16 / 64
reference: sea-ice cost per step	15.8 ms	26.1 ms	43.3 ms	—
wide halo, standard form, ring kernels	−11.4%	−10.0%	−24.3%	—
wide halo, divergence form, ring kernels	+1.9%	−6.1%	−28.4%	—
wide halo, standard form, widened loops	**−57.0%**	**−36.8%**	**−43.4%**	—
wide halo, divergence form, widened loops	**−58.2%**	**−43.3%**	**−50.1%**	—
best widened-loop row, as a share of the model step	−14.4%	−9.6%	−12.7%	−6.8%

NG5's instrumented run was still in the queue at the time of writing; its model-step column is measured and behaves like the others (ring kernels −1.8 and −2.6%, widened loops −5.5 and −6.8%). Two things in the table matter beyond the headline.

It is a compute result, not a communication one. Each pair of rows posts identical message and byte counts — 390.5 messages and 1 542 053 doubles per step for the standard form in both writings. Nothing moved on the wire; the difference is 360 kernel launches. After the rewriting only 2 of the 362 remain on the slow path, the ice-dynamics region falls from 16.5 to 5.5 ms per step, and the *owned* kernels speed up as well, their launch gap falling from 5.5 to 1.3 μs — the constant-memory staging had been serialising its neighbours too.

Your reformulation wins once the launch structure is removed, and we had been measuring past it. With ring kernels the standard form leads by 13.3 points of ice cost on CORE2 and 3.8 on fArc, the divergence form leading only on DARS (4.2). With widened loops the divergence form leads at all four meshes and at every K we tried. It is not a law over all configurations, though: a scaling sweep across node counts (Sect. 5.4) has it ahead at thirteen of the first fourteen points, the exception being fArc on two GPU nodes, where the standard form is

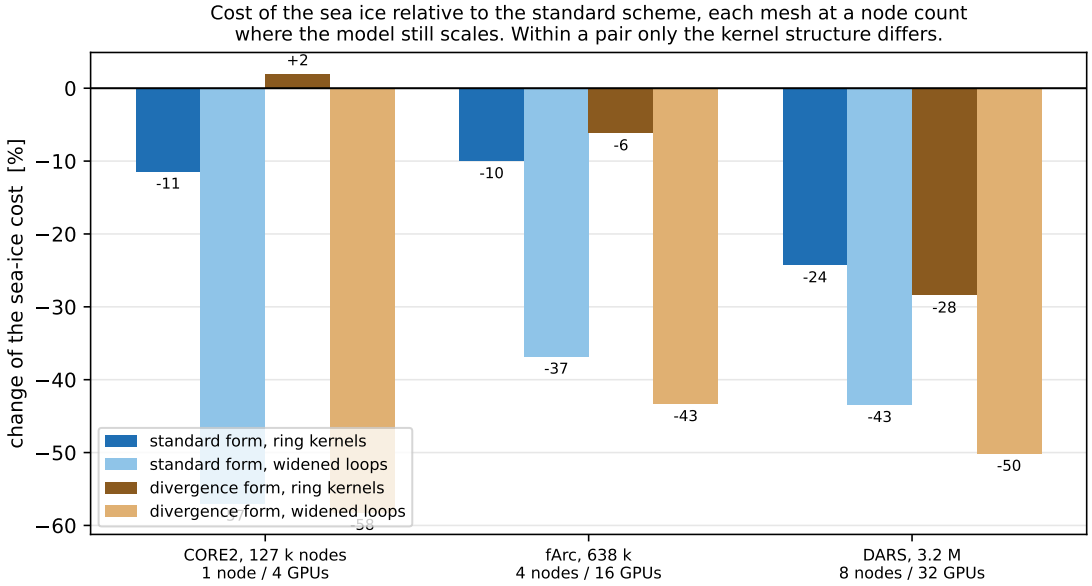

Figure 1: Cost of the sea ice relative to the standard scheme; negative is a saving. Each mesh is at its operating point, the largest node count at which the model still gains from more GPUs. Within each pair of bars the ring, the messages and the bytes are identical — only the kernel structure differs.

0.7 points ahead — outside the repetition spread there, so real. An earlier note of ours decomposed the ring-kernel gap into a message term and a byte term; both were isolated at bit-identical output and both are real, but they were smaller than the launch term sitting on top of them, which fell hardest on the divergence form because its ring gather was the most expensive of the ring kernels.

K**.** Both writings improve monotonically from K=2 to 4 to 8, on both meshes swept and in both forms, so K=8 remains the choice. This was not a foregone conclusion: the term the rewriting removes is launches, which is independent of K, while the ring's arithmetic grows with it.

On CPU the rewriting is not a gain, and the reason is instructive. With no launches to remove it only trades a gather of plain reads for an atomic scatter. In the standard form, where the gather merely re-read stored stresses, that costs 18 to 30% of the ice cost; in the divergence form, where the gather had to recompute each element's stress once per incident ring node, about six times over, it saves 3 to 15%. That the divergence form gains on both backends, while the launch term exists on only one, is the cleanest evidence that this particular term is arithmetic rather than dispatch. Neither writing is profitable on CPU at any of the three core counts tried: the wide halo at least doubles the ice cost there.

5.4 Scaling

Figures 2 and 3 give the strong scaling of the standard scheme and of the two widened-loop wide halos, on both backends, for the model step and for the sea ice separately. The two are shown apart because they have different shapes: the sea ice is where the scheme acts, while the step is dominated by the ocean.

5.5 Does the ring leave the decomposition in the ice?

Giving up the owner's summation order costs what you predicted. After one step of 120 sub-cycles the ice velocity differs from the exact wide halo by 8 to 15 units in the last place; after twenty steps that has grown, by ordinary sensitivity rather than by structure, to the size of this port's own difference between its GPU and CPU arms at identical arithmetic.

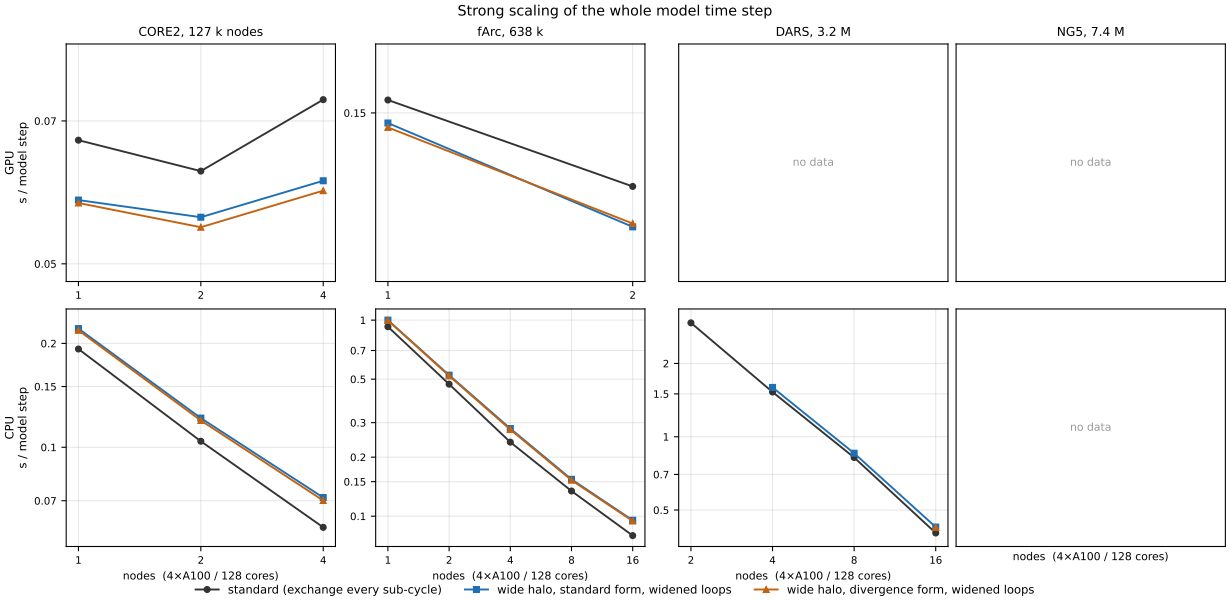

Figure 2: Strong scaling of the whole model time step. Lower is better.

Because that removes the bit-identity check, we hold the widened-loop version to a stronger test instead: 60 days on fArc with 16 GPUs, comparing the mean $|\Delta|$ on the one-cell boundary of each sub-domain with the mean in the interior. A scheme that let halo values age would concentrate its error there, because the decomposition is fixed for the whole run.

boundary / interior ratio, 60 days, fArc, 16 GPUs	two identical runs	ring kernels	**widened loops**
ice thickness	0.854	0.866	**0.836**
ice concentration	0.831	0.817	**0.796**
ice velocity	1.325	1.211	**1.196**

The widened-loop version is at or below its own control on every field, and a placebo built from a decomposition the runs never used gives 0.87 and 0.72, so the ratio does not follow a partition the model never saw. Ice velocity is the field where two identical runs already score 1.33; that is a property of the field on this mesh, which is why each row carries its own control rather than being read against a common threshold.

5.6 What goes on the wire

The wide halo ships eleven auxiliary fields once per model step so that rings $2 \dots K$, which do not exist in FESOM's data structures, have values at all. You objected that this is far more than the sub-cycle needs. We measured both halves of that.

What it costs: 1.6 to 6.2% of the scheme's messages and 3.6 to 9.2% of the data it moves, because it fires once per step against $120/K$ window exchanges.

Whether it is necessary: instrumenting the exchange to compare each arriving owner value against the value the receiving process already holds, **nine of the eleven arrive exactly equal** — ice velocity, concentration, thickness, snow, ocean velocity, wind stress — because FESOM's own halo exchange has already put them there. The two right-hand sides do not, being assembled and area-scaled over owned nodes only. But the redundancy stops at the first ring, and rings $2 \dots K$ have no other source at any K, so what could be removed is 37% of that exchange at $K{=}2$ and 17% at $K{=}4$ — under one percent of the scheme's traffic, and no messages. We did not implement the trimmed version, and that measurement is why.

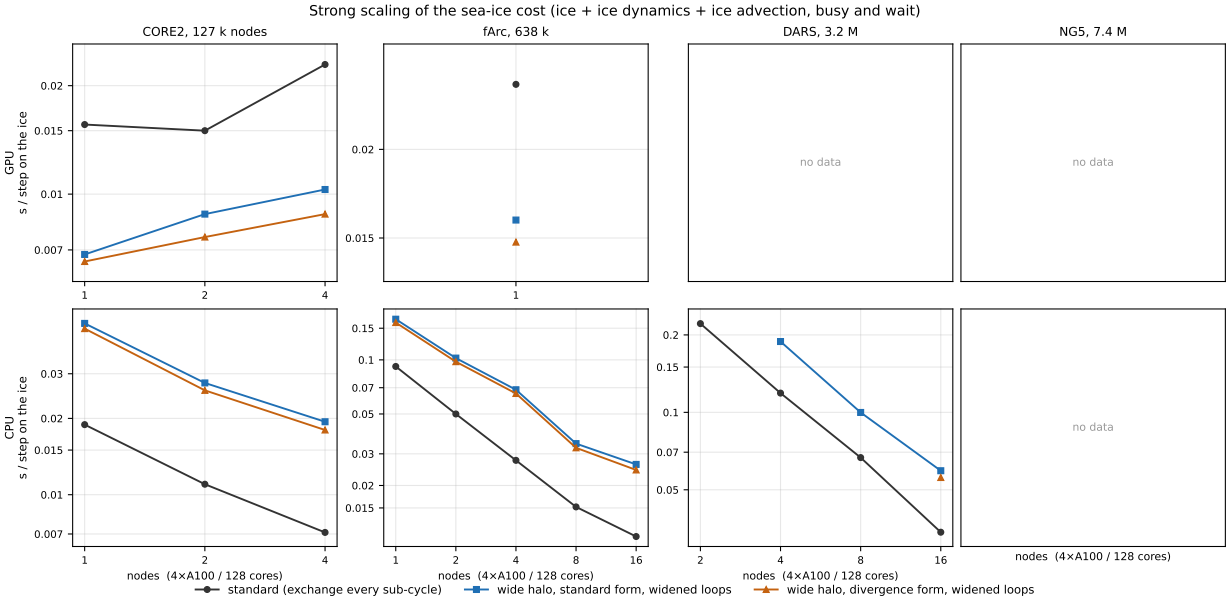

Figure 3: Strong scaling of the sea-ice cost alone — ice, ice dynamics and ice advection, computation together with time spent waiting for communication. This is the measure appropriate to a sea-ice scheme; the model step divides an ice-only change by roughly ten.

6. Summary

	GPU	**CPU**
divergence form alone	+2 to +17% of the ice	−2 to −8.5% of the ice
wide halo, ring kernels	−10 to −28%	not profitable
wide halo, widened loops	**−37 to −58%**	not profitable

For production we would use the wide halo at K=8 on GPU, in the divergence form, written as a widening of the existing loops. Its cost is bit-identity, and the test of Sect. 5.5 is what we would hold it to in place of that. Where bit-identity is required the ring-kernel writing remains available and is exact.

7. Not measured

- **NG5's instrumented run**, so its ice-cost column in Sect. 5.3 is missing; its model step is measured and consistent with the other three.
- **The divergence form over a long integration.** The 2.1×10^{-11} per-step non-equivalence of Sect. 5.1 has not been followed to its long-term consequences.
- **Core counts above 2304 on fArc.** The 2019 paper reaches 6912; the partition set we have stops at 2304, and the separately partitioned copy in the shared archive does not run in this port. Our measured ice share reaches 13% at 2304 cores where the paper interpolates 25–30%; the two candidate reasons — this port's own communication optimisations, and mEVP here versus standard EVP there — are untested.
- **Trimming the eleven auxiliary fields** (Sect. 5.6): measured, and not built.

All configurations are switchable at run time; the underlying runs, job scripts and the full internal write-up can be shared on request.